

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\agent-aea4ba818c0478cd7.jsonl`  
- SHA-256: `a8e8db4e54305aef65054dc132d3e966e26846540b5279e57fe9db0ab276e0a6`  
- Source modified: `2026-06-22T16:12:50+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `subagents`  
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T16:09:56.695Z)

Adversarial code review of a small SPA change in the git repo at
`C:\Users\avidu\Projects\kheIsutra` (a React+TS operator console, "KheIsutra Studio"). Find REAL bugs only ? correctness, security, accessibility regressions, i18n-guard violations, or backward-compat breaks. No style nits, no speculative refactors. Try to break it.

The change adds a "compute picker" to the ingest screen: when the engine reports it can dispatch to a cloud GPU (`/api/capabilities` ? `deployment.cloud\_available === true`), the user can choose "On my computer" (local) vs "In the cloud" (cloud). Cloud requires an explicit cost-acknowledgement checkbox before any submit can run. The choice is sent as `compute` in the POST `/api/process` body. Default is local; when cloud isn't available the picker is hidden and nothing changes.

Ground yourself by reading the diff and the touched files:

1. `git -C C:\Users\avidu\Projects\kheIsutra diff` ? full diff (types.ts, client.ts, useIngestJob.ts, IngestPanel.tsx, IngestPanel.test.tsx, and 6 locale catalogs + their site/public mirror).
2. `web/src/components/IngestPanel.tsx` ? the picker UI, the capabilities `useEffect`, the `cloudBlocked`/`inputsDisabled`/`sendCompute` logic, and how `submit(...)` is called from both the URI form (`onSubmitUri`) and the upload-complete callback (`useUpload((path) => submit(path, locale, sendCompute))`).
3. `web/src/hooks/useUpload.ts` and `web/src/hooks/useIngestJob.ts` ? how the upload-complete callback and submit thread the compute arg. Pay attention to stale-closure risk: does the upload-complete callback capture the LATEST `sendCompute`, or a stale one?
4. `web/src/api/client.ts` `processVideo` ? is `compute` omitted (not sent as null/`) when undefined?
5. The i18n guards: `web/src/i18n/catalog-invariants.test.ts` and `web/src/i18n/hygiene.test.ts`. Verify the new `ingest.compute.\*` strings don't violate: brand-Latin rule (kn/hi must keep Latin "KheIsutra" where en uses it ? my new strings avoid the brand entirely, confirm that's safe), and the engine/server/bucket/backend jargon rule for kn/hi.

Specifically probe and give a verdict on each:

- **Bypass the cost gate**: is there ANY path to dispatch `compute=cloud` WITHOUT the user checking the ack box? Consider: the upload path auto-submits on file-pick (is the file input disabled while cloud is selected+unacked?), the URI form, switching local?cloud after a URI is typed, the disabled-button + early-return belt-and-suspenders.
- **Stale compute on the upload path**: a user picks cloud, acks, picks a file ? does the upload-complete callback send `compute=cloud`? A user picks cloud, acks, switches back to local, then the upload completes ? does it send local or a stale cloud? Trace the ref/closure.
- **Default-OFF / backward compat**: with cloud_available false (or `/api/capabilities` failing), is the ingest flow byte-identical to before (no picker, `compute` omitted from the body)?
- **Graceful degradation**: if `/api/capabilities` rejects or returns garbage, does the component still render and ingest work? Any unhandled promise / crash?
- **Accessibility**: the toggle buttons (aria-pressed), the checkbox label, the 44px tap target (min-h-11) ? any a11y regression vs the existing panel conventions?

- **The `compute` value space**: the SPA sends "local"/"cloud"; confirm the engine treats unknown/None as server-default (you can't see the engine here ? just confirm the SPA only ever sends those two or omits).

Return a concise findings list: severity (blocker/major/minor), exact file:line, why it's real, concrete fix. If nothing real, say so explicitly ? do not invent issues.

2. assistant (2026-06-22T16:09:59.176Z)

I'll start by reading the diff and the touched files to ground myself.

3. user (2026-06-22T16:10:00.889Z)

warning: in the working copy of 'locales/da/common.json', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'locales/en/common.json', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'locales/es/common.json', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'locales/hi/common.json', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'locales/id/common.json', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'locales/kn/common.json', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'site/public/locales/da/common.json', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'site/public/locales/en/common.json', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'site/public/locales/es/common.json', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'site/public/locales/hi/common.json', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'site/public/locales/id/common.json', CRLF will be replaced by LF the next time Git touches it
warning: in the working copy of 'site/public/locales/kn/common.json', CRLF will be replaced by LF the next time Git touches it

```
locales/da/common.json      | 10 +++-
locales/en/common.json      | 10 +++-
locales/es/common.json      | 10 +++-
locales/hi/common.json      | 10 +++-
locales/id/common.json      | 10 +++-
locales/kn/common.json      | 10 +++-
site/public/locales/da/common.json | 10 +++-
site/public/locales/en/common.json | 10 +++-
site/public/locales/es/common.json | 10 +++-
site/public/locales/hi/common.json | 10 +++-
site/public/locales/id/common.json | 10 +++-
site/public/locales/kn/common.json | 10 +++-
web/src/api/client.ts        | 14 +++-
web/src/api/types.ts         | 4 +-
web/src/components/IngestPanel.test.tsx | 98 ++++++
web/src/components/IngestPanel.tsx | 93 ++++++
web/src/hooks/useIngestJob.ts | 4 +-
17 files changed, 304 insertions(+), 29 deletions(-)
---BRANCH---
master
---HEAD---
ba38f15 fix(site): remove dead analytics beacon; use Cloudflare Automatic Setup (#100)
```

4. assistant (2026-06-22T16:10:03.939Z)

Now let me read the full diff and the touched files.

5. user (2026-06-22T16:10:05.312Z)

```
diff --git a/web/src/api/client.ts b/web/src/api/client.ts
index 23c8dd4..f8cabc9 100644
--- a/web/src/api/client.ts
+++ b/web/src/api/client.ts
@@ -82,12 +82,18 @@ @@ async function request<T>(path: string, opts: RequestOpts = {}): Promise<T>
{
  /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host) for
   * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-fails
  400
   * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
  ApiError. */
  -export async function processVideo(videoPath: string, cid?: string, locale?: string):
  Promise<ProcessResponse> {
    - // `locale` (the UI language) is recorded for PMF analytics (which markets use the tool).
    Optional
    - // + omitted when absent (JSON.stringify drops undefined); the engine ignores it until wired.
  +export async function processVideo(
  + videoPath: string,
  + cid?: string,
  + locale?: string,
  + compute?: string,
  +): Promise<ProcessResponse> {
  + // `locale` (the UI language) is recorded for PMF analytics (which markets use the tool).
  + `compute`
  + // is the compute-picker choice ("local" | "cloud") ? where DETECT runs; omitted ? the server
  + // default. Both optional + omitted when absent (JSON.stringify drops undefined).
    return request<ProcessResponse>("/process", {
      method: "POST",
      - body: JSON.stringify({ video_path: videoPath, locale }),
  + body: JSON.stringify({ video_path: videoPath, locale, compute }),
      cid,
    });
  }
diff --git a/web/src/api/types.ts b/web/src/api/types.ts
index 59749ae..37625b9 100644
--- a/web/src/api/types.ts
+++ b/web/src/api/types.ts
@@ -58,7 +58,9 @@ @@ export interface EngineCapabilities {
  /** Whether a Gemini key is PRESENT (never its value) ? drives the AI-highlights step. */
  gemini_key_present?: boolean;
  wasb_weights_present?: boolean;
  - deployment?: { profile?: string; compute_target?: string };
  + /** `cloud_available` = can THIS server satisfy a per-request compute="cloud" dispatch (a
  Cloud Run
  + * GPU Job)? The ingest compute-picker shows the "in the cloud" option only when true. */
  + deployment?: { profile?: string; compute_target?: string; cloud_available?: boolean };
  [key: string]: unknown;
  }
diff --git a/web/src/components/IngestPanel.test.tsx b/web/src/components/IngestPanel.test.tsx
index 113f8e7..ba3fb4d 100644
--- a/web/src/components/IngestPanel.test.tsx
+++ b/web/src/components/IngestPanel.test.tsx
@@ -19,6 +19,9 @@ @@ function fetchSeq(process: unknown, jobs: JobStatus[]): FetchFn {
  let i = 0;
  return vi.fn(async (url: string | URL, init?: RequestInit) => {
    const u = String(url);
  + // The panel probes /api/capabilities on mount (compute-picker) ? answer it benignly so the
  + // ingest-flow assertions don't see a stray "unexpected fetch". No cloud ? no picker.
  + if (u === "/api/capabilities") return httpOk(200, {});
    if (u === "/api/process" && (init?.method ?? "GET") === "POST") return process;
    if (u.startsWith("/api/jobs/")) {
      const body = jobs[Math.min(i, jobs.length - 1)];
```

```

@@ -29,6 +32,32 @@ function fetchSeq(process: unknown, jobs: JobStatus[]): FetchFn {
    }) as unknown as FetchFn;
  }

+/** Like fetchSeq but lets a test set the capabilities payload + capture the /api/process body
+ ? used
+ * to drive and assert the compute-picker. */
+function fetchAll(opts: {
+  caps?: unknown;
+  process?: unknown;
+  jobs?: JobStatus[];
+  onProcessBody?: (body: Record<string, unknown>) => void;
+}): FetchFn {
+  let i = 0;
+  const jobs = opts.jobs ?? [];
+  return vi.fn(async (url: string | URL, init?: RequestInit) => {
+    const u = String(url);
+    if (u === "/api/capabilities") return httpOk(200, opts.caps ?? {});
+    if (u === "/api/process" && (init?.method ?? "GET") === "POST") {
+      opts.onProcessBody?.(JSON.parse(String(init?.body ?? "{}")) as Record<string, unknown>);
+      return opts.process ?? httpOk(202, { job_id: "j", status: "queued", poll: "/api/jobs/j"
    });
+  }
+  if (u.startsWith("/api/jobs/")) {
+    const body = jobs[Math.min(i, jobs.length - 1)];
+    i += 1;
+    return httpOk(200, body);
+  }
+  throw new Error(`unexpected fetch ${u}`);
+}) as unknown as FetchFn;
+}
+
+  beforeEach(async () => {
+    window.localStorage.clear();
+    await i18n.changeLanguage("en"); // guarantees init completed + a known locale for the
assertions
@@ -90,12 +119,75 @@ describe("IngestPanel", () => {
    expect(await screen.findByText(/can't reach khelsutra/i)).toBeTruthy();
  });

-  it("rejects an empty URI with a hint and never calls the engine", () => {
-    const f = vi.fn();
+  it("rejects an empty URI with a hint and never submits to the engine", () => {
+    // The mount-time capabilities probe may fire; assert specifically that /api/process never
does.
+    const f = fetchAll({ caps: {} });
+    vi.stubGlobal("fetch", f);
+    render(<IngestPanel onLoaded={vi.fn()} />);
+    clickProcess();
+    expect(screen.getByTestId("ingest-error")).toBeTruthy();
-    expect(f).not.toHaveBeenCalled();
+    const submitted = (f as unknown as { mock: { calls: unknown[][] } }).mock.calls.some(
+      ([u]) => String(u) === "/api/process",
+    );
+    expect(submitted).toBe(false);
+  });
+
+  // --- compute-picker (item 3)
+  -----
+
+  it("hides the compute picker when the server can't dispatch to the cloud", async () => {
+    vi.stubGlobal("fetch", fetchAll({ caps: { deployment: { cloud_available: false } } }));
+    render(<IngestPanel onLoaded={vi.fn()} />);
+    await act(async () => {}); // let the capabilities probe resolve
+    expect(screen.queryByTestId("compute-picker")).toBeNull();

```

```

+ });
+
+ it("defaults to local and sends compute=local when the cloud is available", async () => {
+   const bodies: Record<string, unknown>[] = [];
+   vi.stubGlobal(
+     "fetch",
+     fetchAll({
+       caps: { deployment: { cloud_available: true } },
+       onProcessBody: (b) => bodies.push(b),
+       jobs: [{ job_id: "j", status: "done", video_id: "vidL", result: { status: "success",
video_id: "vidL" } }],
+     }),
+   );
+   render(<IngestPanel onLoaded={vi.fn()} />);
+   await screen.findByTestId("compute-picker");
+   typeUri("gs://bucket/match.mp4");
+   clickProcess();
+   await act(async () => {});
+   expect(bodies).toHaveLength(1);
+   expect(bodies[0].compute).toBe("local");
+ });
+
+ it("gates cloud on a cost ack, then sends compute=cloud", async () => {
+   const bodies: Record<string, unknown>[] = [];
+   vi.stubGlobal(
+     "fetch",
+     fetchAll({
+       caps: { deployment: { cloud_available: true } },
+       onProcessBody: (b) => bodies.push(b),
+       jobs: [{ job_id: "j", status: "done", video_id: "vidC", result: { status: "success",
video_id: "vidC" } }],
+     }),
+   );
+   render(<IngestPanel onLoaded={vi.fn()} />);
+   await screen.findByTestId("compute-picker");
+   // Type the URI while LOCAL (the input is enabled), then switch to CLOUD.
+   typeUri("gs://bucket/match.mp4");
+   fireEvent.click(screen.getByRole("button", { name: /in the cloud/i }));
+   // Cost note + ack appear; submit is gated until the cost is accepted.
+   expect(screen.getByTestId("compute-cost")).toBeTruthy();
+   expect(screen.getByTestId("compute-ack-required")).toBeTruthy();
+   const submitBtn = screen.getByRole("button", { name: /process video/i }) as
HTMLButtonElement;
+   expect(submitBtn.disabled).toBe(true);
+   clickProcess(); // a disabled button does nothing ? still no submit
+   expect(bodies).toHaveLength(0);
+   // Accept the cost ? the gate lifts ? run on the cloud.
+   fireEvent.click(screen.getByRole("checkbox"));
+   expect(submitBtn.disabled).toBe(false);
+   clickProcess();
+   await act(async () => {});
+   expect(bodies).toHaveLength(1);
+   expect(bodies[0].compute).toBe("cloud");
+ });
+ });
diff --git a/web/src/components/IngestPanel.tsx b/web/src/components/IngestPanel.tsx
index fcebea0..656faae 100644
--- a/web/src/components/IngestPanel.tsx
+++ b/web/src/components/IngestPanel.tsx
@@ -4,14 +4,18 @@
// the SAME back-half: POST /api/process ? poll GET /api/jobs/{id} ? hand the video_id to App.
Honest
// by construction: the engine reports coarse phase WORDS (not a %), and a success-shaped
failure
// (done but no rallies) is shown, never silently treated as a load.

```

```

-import { useState } from "react";
+import { useEffect, useState } from "react";
  import type { ChangeEvent, FormEvent } from "react";
  import { useTranslation } from "react-i18next";
+import { getCapabilities } from "../api/client";
  import { useIngestJob } from "../hooks/useIngestJob";
  import { useUpload } from "../hooks/useUpload";
  import { errorText } from "../lib/errorText";
+import { log } from "../lib/log";
  import { StatusChip, Spinner } from "../StatusChip";

+type Compute = "local" | "cloud";
+
  interface Props {
    /** Called with the engine's video_id once a submitted job finishes successfully. */
    onLoaded: (videoId: string) => void;
@@ -23,13 +27,39 @@ export function IngestPanel({ onLoaded }: Props) {
  const { phase, submit, cancel: cancelIngest, reset: resetIngest, busy: ingestBusy } =
    useIngestJob(onLoaded);
    // The UI locale recorded with the submission (PMF analytics + the localized reel watermark).
    const locale = i18n.resolvedLanguage ?? i18n.language;
+
+  // Compute-picker (PACKAGING compute-picker, item 3): if this server can dispatch to a cloud
+  GPU
+  // (capabilities.deployment.cloud_available), let the user choose where DETECT runs. Default
+  LOCAL
+  // (free, no surprise cost); CLOUD requires an explicit cost acknowledgement before it can
+  run.
+  const [cloudAvailable, setCloudAvailable] = useState(false);
+  const [compute, setCompute] = useState<Compute>("local");
+  const [cloudAck, setCloudAck] = useState(false);
+  useEffect(() => {
+    let alive = true;
+    getCapabilities()
+      .then((c) => alive && setCloudAvailable(c.deployment?.cloud_available === true))
+      .catch((err) => log("warn", "capabilities.unavailable", { error: String(err) }));
+    return () => {
+      alive = false;
+    };
+  }, []);
+  // Only send a choice when the picker is actually shown; otherwise omit it (? server
+  default).
+  const sendCompute = cloudAvailable ? compute : undefined;
+  // CLOUD selected but the cost note not yet accepted ? block submit (don't silently run
+  local, and
+  // don't dispatch a paid job without consent).
+  const cloudBlocked = cloudAvailable && compute === "cloud" && !cloudAck;
+
+  // On upload completion, feed the returned server PATH straight into the engine processing
+  back-half.
-  const upload = useUpload((path) => submit(path, locale));
+  // The arrow is recreated each render (useUpload reads onCompleteRef.current), so sendCompute
+  is fresh.
+  const upload = useUpload((path) => submit(path, locale, sendCompute));
  const busy = ingestBusy || upload.busy;
+  // Disable the submit surfaces (file picker + URI form) while busy OR awaiting the cloud cost
+  ack.
+  const inputsDisabled = busy || cloudBlocked;

  const onSubmitUri = (e: FormEvent) => {
    e.preventDefault();
-    submit(uri, locale);
+    if (cloudBlocked) return;
+    submit(uri, locale, sendCompute);
  };

```

```

    const onFile = (e: ChangeEvent<HTMLInputElement>) => {
      const file = e.target.files?.[0];
@@ -56,6 +86,55 @@ export function IngestPanel({ onLoaded }: Props) {
    <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
    <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>

+    {/* COMPUTE PICKER (item 3): shown only when this server can dispatch to a cloud GPU.
Default
+      LOCAL; CLOUD reveals a cost note + a required acknowledgement before any submit can
run. */}
+    {cloudAvailable && (
+      <fieldset className="mt-4 rounded-xl border border-black/10 p-3" data-testid="compute-
picker">
+        <legend className="px-1 text-xs font-semibold uppercase tracking-wide text-ink/40">
+          {t("ingest.compute.legend")}
+        </legend>
+        <div className="mt-1 flex flex-wrap gap-2">
+          {[["local", "cloud"] as const].map((opt) => (
+            <button
+              key={opt}
+              type="button"
+              aria-pressed={compute === opt}
+              disabled={busy}
+              onClick={() => setCompute(opt)}
+              className={
+                "min-h-11 rounded-xl border px-4 py-2 text-sm font-semibold
disabled:opacity-40 " +
+                (compute === opt
+                  ? "border-green bg-green/10 text-ink"
+                  : "border-black/15 text-ink/70 hover:border-green")
+                }
+              >
+                {t(`ingest.compute.${opt}`)}
+            </button>
+          )]}
+        </div>
+        {compute === "cloud" && (
+          <div className="mt-3" data-testid="compute-cost">
+            <p className="text-xs text-ink/60">{t("ingest.compute.cloudCost")}</p>
+            <label className="mt-2 flex items-center gap-2 text-sm text-ink/80">
+              <input
+                type="checkbox"
+                checked={cloudAck}
+                disabled={busy}
+                onChange={(e) => setCloudAck(e.target.checked)}
+                className="h-4 w-4"
+              />
+              {t("ingest.compute.ack")}
+            </label>
+            {cloudBlocked && (
+              <p className="mt-1 text-xs text-amber-700" data-testid="compute-ack-required">
+                {t("ingest.compute.ackRequired")}
+              </p>
+            )}
+          </div>
+        )}
+      </fieldset>
+    )}

+    {/* PRIMARY: upload a local file (T4). A native file input, visually a button (?44px,
A4). */}
    <div className="mt-4">
      <label
@@ -63,7 +142,7 @@ export function IngestPanel({ onLoaded }: Props) {
      // focus-within makes the visible button show a focus ring when the (sr-only) input

```

```

is
    // focused ? a keyboard user needs a visible focus indicator (WCAG 2.4.7).
    "inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2 text-sm font-
semibold text-white hover:bg-ink2 focus-within:outline-none focus-within:ring-2 focus-
within:ring-green focus-within:ring-offset-2 " +
-   (busy ? "pointer-events-none opacity-40" : "cursor-pointer")
+   (inputsDisabled ? "pointer-events-none opacity-40" : "cursor-pointer")
  }
>
  {t("ingest.chooseFile")}
@@ -71,7 +150,7 @@ export function IngestPanel({ onLoaded }: Props) {
    type="file"
    accept=".mp4,.mov,video/mp4,video/quicktime"
    aria-label={t("ingest.uploadAria")}
-   disabled={busy}
+   disabled={inputsDisabled}
    onChange={onFile}
    className="sr-only"
  />
@@ -89,12 +168,12 @@ export function IngestPanel({ onLoaded }: Props) {
    onChange={e => setUri(e.target.value)}
    placeholder={t("ingest.uriPlaceholder")}
    aria-label={t("ingest.uriAriaLabel")}
-   disabled={busy}
+   disabled={inputsDisabled}
    className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15 bg-white
px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
  />
  <button
    type="submit"
-   disabled={busy}
+   disabled={inputsDisabled}
    className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-white
disabled:opacity-40"
  >
    {phase.status === "submitting" ? t("ingest.submitting") : t("ingest.submit")}
diff --git a/web/src/hooks/useIngestJob.ts b/web/src/hooks/useIngestJob.ts
index b308ba2..2defe4c 100644
--- a/web/src/hooks/useIngestJob.ts
+++ b/web/src/hooks/useIngestJob.ts
@@ -49,7 +49,7 @@ export function useIngestJob(onLoaded?: (videoId: string) => void) {
  useEffect(() => stop, [stop]);

  const submit = useCallback(
-   (rawUri: string, locale?: string) => {
+   (rawUri: string, locale?: string, compute?: string) => {
    const uri = rawUri.trim();
    if (!uri) {
      setPhase({ status: "error", error: { code: "invalidUri" } });
@@ -116,7 +116,7 @@ export function useIngestJob(onLoaded?: (videoId: string) => void) {
  void (async () => {
    let jobId: string;
    try {
-     const resp = await processVideo(uri, cid, locale);
+     const resp = await processVideo(uri, cid, locale, compute);
    jobId = resp.job_id;
  } catch (err) {
    fail(classifyError(err), "ingest.submit_error");

```

6. assistant (2026-06-22T16:10:07.968Z)

Now let me read the key source files in full to verify the logic, especially the upload hook's stale-closure behavior.

7. user (2026-06-22T16:10:08.934Z)

```

1      // The front-half ingest screen (appliance S1/S2; tracker B-P2 + T4). Two ways to point
the tool at a
2      // video: UPLOAD a local file (primary ? alpha users have a file, not a URI; chunked to
the engine's
3      // /api/upload/* endpoints) or paste a `gs://` bucket URI / host path (secondary). Both
converge on
4      // the SAME back-half: POST /api/process ? poll GET /api/jobs/{id} ? hand the video_id
to App. Honest
5      // by construction: the engine reports coarse phase WORDS (not a %), and a success-
shaped failure
6      // (done but no rallies) is shown, never silently treated as a load.
7      import { useEffect, useState } from "react";
8      import type { ChangeEvent, FormEvent } from "react";
9      import { useTranslation } from "react-i18next";
10     import { getCapabilities } from "../api/client";
11     import { useIngestJob } from "../hooks/useIngestJob";
12     import { useUpload } from "../hooks/useUpload";
13     import { errorText } from "../lib/errorText";
14     import { log } from "../lib/log";
15     import { StatusChip, Spinner } from "../StatusChip";
16
17     type Compute = "local" | "cloud";
18
19     interface Props {
20       /** Called with the engine's video_id once a submitted job finishes successfully. */
21       onLoaded: (videoId: string) => void;
22     }
23
24     export function IngestPanel({ onLoaded }: Props) {
25       const { t, i18n } = useTranslation();
26       const [uri, setUri] = useState("");
27       const { phase, submit, cancel: cancelIngest, reset: resetIngest, busy: ingestBusy } =
useIngestJob(onLoaded);
28       // The UI locale recorded with the submission (PMF analytics + the localized reel
watermark).
29       const locale = i18n.resolvedLanguage ?? i18n.language;
30
31       // Compute-picker (PACKAGING compute-picker, item 3): if this server can dispatch to a
cloud GPU
32       // (capabilities.deployment.cloud_available), let the user choose where DETECT runs.
Default LOCAL
33       // (free, no surprise cost); CLOUD requires an explicit cost acknowledgement before it
can run.
34       const [cloudAvailable, setCloudAvailable] = useState(false);
35       const [compute, setCompute] = useState<Compute>("local");
36       const [cloudAck, setCloudAck] = useState(false);
37       useEffect(() => {
38         let alive = true;
39         getCapabilities()
40           .then((c) => alive && setCloudAvailable(c.deployment?.cloud_available === true))
41           .catch((err) => log("warn", "capabilities.unavailable", { error: String(err) }));
42         return () => {
43           alive = false;
44         };
45       }, []);
46       // Only send a choice when the picker is actually shown; otherwise omit it (? server
default).
47       const sendCompute = cloudAvailable ? compute : undefined;
48       // CLOUD selected but the cost note not yet accepted ? block submit (don't silently
run local, and
49       // don't dispatch a paid job without consent).
50       const cloudBlocked = cloudAvailable && compute === "cloud" && !cloudAck;
51
52       // On upload completion, feed the returned server PATH straight into the engine
processing back-half.

```

```

53      // The arrow is recreated each render (useUpload reads onCompleteRef.current), so
sendCompute is fresh.
54      const upload = useUpload((path) => submit(path, locale, sendCompute));
55      const busy = ingestBusy || upload.busy;
56      // Disable the submit surfaces (file picker + URI form) while busy OR awaiting the
cloud cost ack.
57      const inputsDisabled = busy || cloudBlocked;
58
59      const onSubmitUri = (e: FormEvent) => {
60        e.preventDefault();
61        if (cloudBlocked) return;
62        submit(uri, locale, sendCompute);
63      };
64      const onFile = (e: ChangeEvent<HTMLInputElement>) => {
65        const file = e.target.files?.[0];
66        if (file) upload.start(file);
67        e.target.value = ""; // let re-picking the same file fire change again
68      };
69      const cancelAll = () => {
70        upload.cancel();
71        cancelIngest();
72      };
73      const resetAll = () => {
74        upload.reset();
75        resetIngest();
76      };
77
78      // Shared mapping with the build flow (lib/errorText): an `engine` failure carries the
engine's own
79      // message (a 400 scheme/path/extension detail, a worker error); every other code maps
to a friendly
80      // localized line. Surface whichever half failed (upload or engine) ? they're mutually
exclusive.
81      const errorState = upload.phase.status === "error" ? upload.phase.error : phase.status
=== "error" ? phase.error : null;
82      const errorMsg = errorState ? errorText(t, errorState) : null;
83
84      return (
85        <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
86          <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
87          <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
88
89          {/* COMPUTE PICKER (item 3): shown only when this server can dispatch to a cloud
GPU. Default
90             LOCAL; CLOUD reveals a cost note + a required acknowledgement before any
submit can run. */}
91          {cloudAvailable && (
92            <fieldset className="mt-4 rounded-xl border border-black/10 p-3" data-
testid="compute-picker">
93              <legend className="px-1 text-xs font-semibold uppercase tracking-wide text-
ink/40">
94                {t("ingest.compute.legend")}
95              </legend>
96              <div className="mt-1 flex flex-wrap gap-2">
97                {[["local", "cloud"] as const].map((opt) => (
98                  <button
99                    key={opt}
100                    type="button"
101                    aria-pressed={compute === opt}
102                    disabled={busy}
103                    onClick={() => setCompute(opt)}
104                    className={
105                      "min-h-11 rounded-xl border px-4 py-2 text-sm font-semibold
disabled:opacity-40 " +

```

```

106         (compute === opt
107         ? "border-green bg-green/10 text-ink"
108         : "border-black/15 text-ink/70 hover:border-green")
109     }
110 >
111     {t(`ingest.compute.${opt}`)}
112 </button>
113 )}}
114 </div>
115 {compute === "cloud" && (
116     <div className="mt-3" data-testid="compute-cost">
117         <p className="text-xs text-ink/60">{t("ingest.compute.cloudCost")}</p>
118         <label className="mt-2 flex items-center gap-2 text-sm text-ink/80">
119             <input
120                 type="checkbox"
121                 checked={cloudAck}
122                 disabled={busy}
123                 onChange={(e) => setCloudAck(e.target.checked)}
124                 className="h-4 w-4"
125             />
126             {t("ingest.compute.ack")}
127         </label>
128         {cloudBlocked && (
129             <p className="mt-1 text-xs text-amber-700" data-testid="compute-ack-
required">
130                 {t("ingest.compute.ackRequired")}
131             </p>
132         )}
133     </div>
134 )}
135 </fieldset>
136 )}
137
138 {/* PRIMARY: upload a local file (T4). A native file input, visually a button
(?44px, A4). */}
139 <div className="mt-4">
140     <label
141         className={
142             // focus-within makes the visible button show a focus ring when the (sr-
only) input is
143             // focused ? a keyboard user needs a visible focus indicator (WCAG 2.4.7).
144             "inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2 text-sm font-
semibold text-white hover:bg-ink2 focus-within:outline-none focus-within:ring-2 focus-
within:ring-green focus-within:ring-offset-2 " +
145             (inputsDisabled ? "pointer-events-none opacity-40" : "cursor-pointer")
146         }
147     >
148         {t("ingest.chooseFile")}
149         <input
150             type="file"
151             accept=".mp4,.mov,video/mp4,video/quicktime"
152             aria-label={t("ingest.uploadAria")}
153             disabled={inputsDisabled}
154             onChange={onFile}
155             className="sr-only"
156         />
157     </label>
158     <p className="mt-2 text-xs text-ink/40">{t("ingest.fileHint")}</p>
159 </div>
160
161 {/* SECONDARY: or point at a cloud bucket / host path. */}
162 <div className="mt-4 border-t border-black/5 pt-4">
163     <p className="text-xs font-semibold uppercase tracking-wide text-
ink/40">{t("ingest.orLink")}</p>
164     <form onSubmit={onSubmitUri} className="mt-2 flex flex-wrap items-center gap-2">

```

```

165         <input
166             type="text"
167             value={uri}
168             onChange={(e) => setUri(e.target.value)}
169             placeholder={t("ingest.uriPlaceholder")}
170             aria-label={t("ingest.uriAriaLabel")}
171             disabled={inputsDisabled}
172             className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
173         />
174         <button
175             type="submit"
176             disabled={inputsDisabled}
177             className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
178         >
179             {phase.status === "submitting" ? t("ingest.submitting") :
t("ingest.submit")}
180         </button>
181     </form>
182     <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
183 </div>
184
185     {/* Live status (aria-live so screen readers hear progress) as icon+text chips ?
never icon-only
186         (A8). Upload shows a determinate % bar; the engine phase is indeterminate
(phase words). */}
187     <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
188         {upload.phase.status === "uploading" && (
189             <div data-testid="upload-progress">
190                 {/* Static chip label (not the per-tick %) so the aria-live region announces
once; the live
191                     % rides the progressbar's aria-valuenow + a sighted-only label (A8/T4).
*/}
192                 <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.uploadingStatus")} />
193                 <div className="mt-2 flex max-w-sm items-center gap-2">
194                     <div
195                         className="h-1.5 flex-1 overflow-hidden rounded-full bg-black/10"
196                         role="progressbar"
197                         aria-label={t("ingest.uploadProgress")}
198                         aria-valuenow={upload.phase.pct}
199                         aria-valuemin={0}
200                         aria-valuemax={100}
201                     >
202                         <div className="h-full bg-green transition-all" style={{ width:
`${upload.phase.pct}%` }} />
203                     </div>
204                     <span aria-hidden="true" className="text-xs tabular-nums text-ink/50">
{upload.phase.pct}%
205                     </span>
206                 </div>
207             </div>
208             <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
209             <button
210                 type="button"
211                 onClick={cancelAll}
212                 className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
213             >
214                 {t("ingest.cancel")}
215             </button>
216         </div>
217     )}
218     {ingestBusy && (
219         <div>

```

```

220         <StatusChip
221             tone="info"
222             icon={<Spinner />}
223             label={
224                 t("ingest.working") +
225                 (phase.status === "polling" && phase.progress
226                     ? ` · ${t("ingest.progress", { progress: phase.progress })}`
227                     : "")
228             }
229         />
230         {/* Reassurance for slow uplinks: the job runs on the server, so it's safe
to wait. */}
231         <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
232         <button
233             type="button"
234             onClick={cancelAll}
235             className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
236             >
237             {t("ingest.cancel")}
238         </button>
239     </div>
240 )}
241 {errorMsg && (
242     <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
243         <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />
244         <p className="mt-2">{errorMsg}</p>
245         <button
246             type="button"
247             onClick={resetAll}
248             className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
249             >
250             {t("ingest.retry")}
251         </button>
252     </div>
253 )}
254 </div>
255 </section>
256 );
257 }
258

```

8. user (2026-06-22T16:10:09.426Z)

```

1     // Drives the in-UI file UPLOAD half of ingest (tracker T4): chunk a local file up to
the engine's
2     // /api/upload/* endpoints, report byte progress, then hand the returned server PATH to
the existing
3     // processing back-half (useIngestJob.submit). Alpha users have a FILE, not a gs:// URI
? this is the
4     // keystone that lets them point the tool at it from the browser.
5     //
6     // Kept i18n-FREE (returns stable IngestErrorCodes via classifyError, not prose) so it
unit-tests
7     // without a translation context ? IngestPanel maps codes to localized strings via
errorText. Abort
8     // is an AbortController: cancel/unmount/a superseding pick stops the in-flight upload
and the
9     // orchestrator best-effort DELETES the server-side partial.
10    import { useCallback, useEffect, useRef, useState } from "react";
11    import { uploadFile } from "../api/client";
12    import { classifyError, type IngestError } from "../lib/errors";
13    import { log, newCorrelationId } from "../lib/log";

```

```

14
15     export type UploadPhase =
16       | { status: "idle" }
17       | { status: "uploading"; pct: number; filename: string }
18       | { status: "error"; error: IngestError };
19
20     export function useUpload(onComplete: (path: string) => void) {
21       const [phase, setPhase] = useState<UploadPhase>({ status: "idle" });
22       const ctrlRef = useRef<AbortController | null>(null);
23       // Keep the latest onComplete without making start depend on it (a parent re-render
won't restart).
24       const onCompleteRef = useRef(onComplete);
25       onCompleteRef.current = onComplete;
26
27       const stop = useCallback(() => {
28         ctrlRef.current?.abort();
29         ctrlRef.current = null;
30       }, []);
31
32       // Abort any in-flight upload on unmount.
33       useEffect(() => stop, [stop]);
34
35       const start = useCallback(
36         (file: File) => {
37           // A 0-byte "video" is never valid ? reject before touching the network (reuse
invalidUri copy).
38           if (!file || file.size === 0) {
39             setPhase({ status: "error", error: { code: "invalidUri" } });
40             return;
41           }
42
43           stop(); // supersede any prior upload
44           const ctrl = new AbortController();
45           ctrlRef.current = ctrl;
46           const cid = newCorrelationId();
47           setPhase({ status: "uploading", pct: 0, filename: file.name });
48           log("info", "upload.submit_start", { cid, filename: file.name, size: file.size });
49
50           void (async () => {
51             try {
52               const res = await uploadFile(file, {
53                 cid,
54                 signal: ctrl.signal,
55                 onProgress: ({ sentBytes, totalBytes }) => {
56                   if (ctrl.signal.aborted) return;
57                   const pct = totalBytes > 0 ? Math.min(100, Math.round((sentBytes /
totalBytes) * 100)) : 0;
58                   setPhase({ status: "uploading", pct, filename: file.name });
59                 },
60               });
61               if (ctrl.signal.aborted) return;
62               setPhase({ status: "idle" }); // hand off ? the engine processing phase takes
over
63               onCompleteRef.current(res.path);
64             } catch (err) {
65               // User-cancelled (abort) is silent, not an error.
66               if (ctrl.signal.aborted || (err instanceof DOMException && err.name ===
"AbortError")) return;
67               setPhase({ status: "error", error: classifyError(err) });
68               log("error", "upload.failed", { cid, filename: file.name, error: String(err)
});
69             }
70           })();
71         },
72         [stop],

```

```

73     );
74
75     const cancel = useCallback(() => {
76         stop();
77         setPhase({ status: "idle" });
78         log("info", "upload.cancel", {});
79     }, [stop]);
80
81     const reset = useCallback(() => {
82         stop();
83         setPhase({ status: "idle" });
84     }, [stop]);
85
86     const busy = phase.status === "uploading";
87     return { phase, start, cancel, reset, busy };
88 }
89

```

9. user (2026-06-22T16:10:09.762Z)

```

1      // Drives the front-half ingest: submit a video to POST /api/process, then poll GET
/api/jobs/{id}
2      // until it finishes, and hand the resulting video_id to the existing rally-loading
back-half.
3      //
4      // Kept i18n-FREE (returns stable IngestErrorCodes, not prose) so it unit-tests without
a
5      // translation context ? IngestPanel maps codes to localized strings. The poll loop is a
RECURSIVE
6      // setTimeout (never setInterval): each tick waits for the prior getJob to resolve, so a
slow engine
7      // can't pile up overlapping requests. It backs off, gives up after a ceiling, and is
fully
8      // cancellable on unmount / user-cancel / a superseding submit.
9      import { useCallback, useEffect, useRef, useState } from "react";
10     import { getJob, processVideo } from "../api/client";
11     import { classifyError, type IngestError } from "../lib/errors";
12     import { log, newCorrelationId } from "../lib/log";
13     import type { JobStatus } from "../api/types";
14
15     const POLL_MIN_MS = 1000;
16     const POLL_MAX_MS = 5000;
17     const POLL_BACKOFF = 1.5;
18     const POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' job won't poll forever
19
20     export type IngestPhase =
21     | { status: "idle" }
22     | { status: "submitting" }
23     | { status: "polling"; progress: string | null }
24     | { status: "done"; videoId: string }
25     | { status: "error"; error: IngestError };
26
27     interface Run {
28         cancelled: boolean;
29         timer?: ReturnType<typeof setTimeout>;
30     }
31
32     export function useIngestJob(onLoaded?: (videoId: string) => void) {
33         const [phase, setPhase] = useState<IngestPhase>({ status: "idle" });
34         const runRef = useRef<Run | null>(null);
35         // Keep the latest onLoaded without making submit depend on it (a parent re-render
won't restart polls).
36         const onLoadedRef = useRef(onLoaded);
37         onLoadedRef.current = onLoaded;
38

```

```

39     const stop = useCallback(() => {
40         const run = runRef.current;
41         if (run) {
42             run.cancelled = true;
43             if (run.timer) clearTimeout(run.timer);
44         }
45         runRef.current = null;
46     }, []);
47
48     // Cancel any in-flight poll loop when the component unmounts (App.tsx:80-82 alive-
flag idiom).
49     useEffect(() => stop, [stop]);
50
51     const submit = useCallback(
52         (rawUri: string, locale?: string, compute?: string) => {
53             const uri = rawUri.trim();
54             if (!uri) {
55                 setPhase({ status: "error", error: { code: "invalidUri" } });
56                 return;
57             }
58
59             stop(); // supersede any prior run
60             const run: Run = { cancelled: false };
61             runRef.current = run;
62
63             const cid = newCorrelationId();
64             const startedAt = Date.now();
65             setPhase({ status: "submitting" });
66             log("info", "ingest.submit_start", { cid, uri });
67
68             const fail = (error: IngestError, event: string) => {
69                 if (run.cancelled) return;
70                 setPhase({ status: "error", error });
71                 const level = error.code === "timeout" || error.code === "emptyResult" ? "warn"
: "error";
72                 log(level, event, { cid, uri, code: error.code, detail: error.detail });
73             };
74
75             const poll = (jobId: string, delay: number) => {
76                 run.timer = setTimeout(() => {
77                     void (async () => {
78                         if (run.cancelled) return;
79                         let job: JobStatus;
80                         try {
81                             job = await getJob(jobId, cid);
82                         } catch (err) {
83                             fail(classifyError(err), "ingest.poll_error");
84                             return;
85                         }
86                         if (run.cancelled) return;
87
88                         if (job.status === "done") {
89                             const videoId = job.video_id ?? job.result?.video_id ?? null;
90                             // Two status layers: a `done` job with a `failed` pipeline verdict (or no
video_id) is
91                             // a success-shaped FAILURE ? never load it, never hang.
92                             if (job.result?.status === "failed" || !videoId) {
93                                 fail({ code: "emptyResult" }, "ingest.empty_result");
94                                 return;
95                             }
96                             setPhase({ status: "done", videoId });
97                             log("info", "ingest.done", { cid, uri, video_id: videoId });
98                             onLoadedRef.current?.(videoId);
99                             return;
100                         }

```

```

101         if (job.status === "failed") {
102             fail({ code: "engine", detail: job.error ?? "job failed" },
"ingest.job_failed");
103             return;
104         }
105         // queued | running ? reflect coarse progress (free-text phase) and keep
polling.
106         setPhase({ status: "polling", progress: job.progress ?? null });
107         if (Date.now() - startedAt > POLL_CEILING_MS) {
108             fail({ code: "timeout" }, "ingest.poll_timeout");
109             return;
110         }
111         poll(jobId, Math.min(delay * POLL_BACKOFF, POLL_MAX_MS));
112     }());
113 }, delay);
114 };
115
116 void (async () => {
117     let jobId: string;
118     try {
119         const resp = await processVideo(uri, cid, locale, compute);
120         jobId = resp.job_id;
121     } catch (err) {
122         fail(classifyError(err), "ingest.submit_error");
123         return;
124     }
125     if (run.cancelled) return;
126     log("info", "ingest.queued", { cid, uri, job_id: jobId });
127     setPhase({ status: "polling", progress: null });
128     poll(jobId, POLL_MIN_MS);
129 }());
130 },
131 [stop],
132 );
133
134 const cancel = useCallback(() => {
135     stop();
136     setPhase({ status: "idle" });
137     log("info", "ingest.cancel", {});
138 }, [stop]);
139
140 const reset = useCallback(() => {
141     stop();
142     setPhase({ status: "idle" });
143 }, [stop]);
144
145 const busy = phase.status === "submitting" || phase.status === "polling";
146 return { phase, submit, cancel, reset, busy };
147 }
148

```

10. assistant (2026-06-22T16:10:14.130Z)

Now let me examine the critical stale-closure path. The `useUpload` hook's `onComplete` arrow is `(path) => submit(path, locale, sendCompute)`. Let me verify `sendCompute` is captured fresh by checking how `onCompleteRef.current` is updated, and trace the disabled-state logic carefully. Let me also read the client.ts and the i18n guard tests and locale catalogs.

11. user (2026-06-22T16:10:15.122Z)

```

1      // Typed client for the engine FastAPI. Default = SAME-ORIGIN relative `/api/*` (the
single-origin
2      // appliance reverse-proxies UI + engine behind one origin; in dev Vite proxies `/api` ?
:8000).
3      // Every call carries a correlation id (header + logs) so a journey is traceable end to

```

```

end.
4   import { log, newCorrelationId } from "../lib/log";
5   import type {
6     RallySegment,
7     QueryResponse,
8     CompileResponse,
9     EngineConfig,
10    EngineCapabilities,
11    ProcessResponse,
12    JobStatus,
13    UploadInitResponse,
14    UploadPartResponse,
15    UploadCompleteResponse,
16  } from "./types";
17
18  /** Resolve the engine API base. Default same-origin `"/api"`. For a SPLIT deploy ? SPA
on
19    * Cloudflare Pages + the engine on a separate `api.*` cloudflared tunnel
(HOSTING_PLAN.md §2) ?
20    * set `VITE_API_BASE` at build time to the engine's API base **including the `/api`
path**
21    * (e.g. `https://api.khelsutra.guru/api`), and lock the engine's
`RALLY_CORS_ALLOW_ORIGINS` to the
22    * Pages origin + put CF Access in front of both hostnames. A trailing slash is trimmed
so
23    * `` `${base}/process` `` never double-slashes. */
24  export function apiBase(): string {
25    return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
26  }
27
28  const BASE = apiBase();
29
30  export class ApiError extends Error {
31    readonly cid?: string;
32    readonly status?: number;
33    readonly body?: string;
34    constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
35      super(message);
36      this.name = "ApiError";
37      this.cid = opts.cid;
38      this.status = opts.status;
39      this.body = opts.body;
40      if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
41    }
42  }
43
44  interface RequestOpts {
45    method?: string;
46    body?: string;
47    headers?: Record<string, string>;
48    /** Correlation id to thread through this call; auto-generated if absent. */
49    cid?: string;
50  }
51
52  async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
53    const cid = opts.cid ?? newCorrelationId();
54    const url = `${BASE}${path}`;
55    const method = opts.method ?? "GET";
56    log("info", "api.request", { cid, method, url });
57
58    let res: Response;
59    try {
60      res = await fetch(url, {
61        method,

```

```

62         body: opts.body,
63         headers: { "content-type": "application/json", "x-correlation-id": cid,
...(opts.headers ?? {}) },
64     });
65     } catch (err) {
66         // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
67         log("error", "api.network_error", { cid, method, url, error: String(err) });
68         throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
69     }
70
71     if (!res.ok) {
72         const body = await res.text().catch(() => "");
73         log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
74         throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
75     }
76
77     const data = (await res.json()) as T;
78     log("info", "api.response", { cid, method, url, status: res.status });
79     return data;
80 }
81
82 /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
83 * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
fails 400
84 * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
85 export async function processVideo(
86     videoPath: string,
87     cid?: string,
88     locale?: string,
89     compute?: string,
90 ): Promise<ProcessResponse> {
91     // `locale` (the UI language) is recorded for PMF analytics (which markets use the
tool). `compute`
92     // is the compute-picker choice ("local" | "cloud") ? where DETECT runs; omitted ? the
server
93     // default. Both optional + omitted when absent (JSON.stringify drops undefined).
94     return request<ProcessResponse>("/process", {
95         method: "POST",
96         body: JSON.stringify({ video_path: videoPath, locale, compute }),
97         cid,
98     });
99 }
100
101 /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
102 export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
103     return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
104 }
105
106 /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
107 export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
108     const data = await request<QueryResponse>("/query", {
109         method: "POST",
110         body: JSON.stringify({ video_id: videoId }),
111         cid,
112     });
113     return data.play_segments ?? [];
114 }
115

```

```

116    /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,
output_filename} [main.py:342]. */
117    export async function compileReel(
118        videoId: string,
119        segmentIds: number[],
120        outputFilename: string,
121        cid?: string,
122        locale?: string,
123    ): Promise<CompileResponse> {
124        // `locale` (the UI language) localizes the reel's branding watermark engine-side.
Optional +
125        // omitted when absent; the engine falls back to the default watermark for an
unknown/absent locale.
126        return request<CompileResponse>("/compile", {
127            method: "POST",
128            body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename, locale }),
129            cid,
130        });

```

12. user (2026-06-22T16:10:16.085Z)

```

1    // Content regression guards for the localization catalogs ? the rules the adversarial
translation
2    // audit (#77) established, pinned so the specific bugs can never silently return. This
complements
3    // hygiene.test.ts (which proves key/placeholder PARITY) by asserting catalog CONTENT.
Each block maps
4    // to a concrete #77 bug class; the mutation that reintroduces the bug fails here. Pure
data assertions
5    // over repo-root /locales (D-CATALOG-HOME) ? no render context needed.
6    import { describe, it, expect } from "vitest";
7    import { readFileSync, readdirSync } from "node:fs";
8    import { join } from "node:path";
9
10   const LOCALES_DIR = join(import.meta.dirname, "..", "..", "..", "locales");
11
12   type Flat = Record<string, string>;
13   function flatEntries(obj: Record<string, unknown>, prefix = ""): Flat {
14       const out: Flat = {};
15       for (const [k, v] of Object.entries(obj)) {
16           const key = prefix ? `${prefix}.${k}` : k;
17           if (v && typeof v === "object" && !Array.isArray(v)) Object.assign(out,
flatEntries(v as Record<string, unknown>, key));
18           else out[key] = String(v);
19       }
20       return out;
21   }
22   const load = (loc: string): Flat =>
23       flatEntries(JSON.parse(readFileSync(join(LOCALES_DIR, loc, "common.json"), "utf8")) as
Record<string, unknown>);
24
25   const ALL_LOCALES = readdirSync(LOCALES_DIR, { withFileTypes: true })
26       .filter((d) => d.isDirectory())
27       .map((d) => d.name);
28   const en = load("en");
29
30   // (1) Internal-token / debug-tag leaks ? EVERY locale. The engine's internal
`min_shot_count` config
31   // name leaked into footer.floorSomeRest (#77, kn even wrapped it in <code>). No
developer/debug tag
32   // (kbd/samp/tt/var) or that token may reach any user-facing string. <code> is likewise
banned ?
33   // EXCEPT on the marketing site's developer "own your model" page (site.ownModel.*),
which intentionally

```

```

34 // renders inline <code> (the rally_annotator.lua filename, the ending_reason enum,
<video>.rallies.csv)
35 // through the site shim's data-i18n-html form. That namespace is the sole, deliberate
exception; the
36 // SPA console keys (where the #77 leak happened) still fail on any <code>.
37 describe("catalog content ? no internal token / debug-tag leaks (every locale)", () => {
38     const TOKEN = /min_shot_count/;
39     const DEBUG_TAG = /<\/?(?:kbd|samp|tt|var)\b/i;
40     const CODE_TAG = /<\/?code\b/i;
41     const codeAllowed = (key: string) => key.startsWith("site.ownModel.");
42     for (const loc of ALL_LOCALES) {
43         it(`${loc}: no 'min_shot_count' token or stray <code>/<kbd>/<samp>/<tt>/<var> tag`,
() => {
44             const offenders = Object.entries(load(loc))
45                 .filter(([k, v]) => TOKEN.test(v) || DEBUG_TAG.test(v) || (CODE_TAG.test(v) &&
!codeAllowed(k)))
46                 .map(([k]) => k);
47             expect(offenders).toEqual([]);
48         });
49     }
50 });
51
52 // (2) Brand ? never transliterated, and present (Latin) on EVERY key en uses it on. The
positive rule
53 // is the real guard: self-maintaining (auto-covers future brand keys, incl. errors.* /
ingest.hint that
54 // the old 3-key denylist missed) and it catches ANY transliteration spelling at once ?
a transliterated
55 // value simply won't contain Latin "KheI Sutra". The Indic-spelling denylist is cheap
defence-in-depth.
56 describe("catalog content ? the brand stays Latin 'KheI Sutra' everywhere en uses it", ()
=> {
57     const brandKeys = Object.keys(en).filter((k) => en[k].includes("KheI Sutra"));
58
59     it("en actually carries the brand on several keys (so the guard is meaningful)", () =>
{
60         expect(brandKeys.length).toBeGreaterThanOrEqual(6);
61     });
62
63     for (const loc of ["kn", "hi"] as const) {
64         it(`${loc}: Latin "KheI Sutra" is present on every key whose en value uses it`, () =>
{
65             const e = load(loc);
66             const missing = brandKeys.filter((k) => !(e[k] ?? "").includes("KheI Sutra"));
67             expect(missing).toEqual([]);
68         });
69     }
70
71     const TRANSLIT: Record<string, string> = { kn: "?????????", hi: "?????????" };
72     for (const loc of ["kn", "hi"] as const) {
73         it(`${loc}: the known Indic transliteration of the brand never appears`, () => {
74             const offenders = Object.entries(load(loc))
75                 .filter(([k, v]) => v.includes(TRANSLIT[loc]))
76                 .map(([k]) => k);
77             expect(offenders).toEqual([]);
78         });
79     }
80 });
81
82 // (3) Engine/server/bucket/backend jargon only where en uses it (kn/hi). Self-
maintaining: a localized
83 // jargon term ? the Latin word OR an Indic transliteration ? is allowed ONLY on a key
whose en value
84 // also uses the English word. en avoids these in body copy (uses the brand / "cloud-
storage" / passive

```

```

85 // voice) EXCEPT the technical ingest.uriAriaLabel + uriPlaceholder; #77 leaked them
into kn/hi elsewhere.
86 describe("catalog content ? engine/server/bucket jargon only where en uses it (kn/hi)",
() => {
87     const ENGLISH = ["engine", "server", "bucket", "backend"] as const;
88     const TRANSLIT: Record<string, Record<(typeof ENGLISH)[number], string[]>> = {
89         kn: { engine: ["??????"], server: ["???????", "????????"], bucket: ["??????"], backend:
["????????????"] },
90         hi: { engine: ["??????"], server: ["???????", "??????????"], bucket: ["??????"], backend:
["?????????"] },
91     };
92     for (const loc of ["kn", "hi"] as const) {
93         it(`${loc}: a jargon term (Latin or transliterated) appears only where en uses the
English word`, () => {
94             const e = load(loc);
95             const violations: Array<{ key: string; term: string; en: string }> = [];
96             for (const [key, val] of Object.entries(e)) {
97                 const enUsesWord = (w: string) => new RegExp(`\\b${w}\\b`, "i").test(en[key] ??
""));
98                 for (const w of ENGLISH) {
99                     const present = [
100                         ...(new RegExp(`\\b${w}\\b`, "i").test(val) ? [w] : []),
101                         ...TRANSLIT[loc][w].filter((t) => val.includes(t)),
102                     ];
103                     for (const term of present) if (!enUsesWord(w)) violations.push({ key, term,
en: en[key] ?? "(absent)" });
104                 }
105             }
106             expect(violations).toEqual([]);
107         });
108     }
109 });
110
111 // (4) Hindi de-Sanskritization (#77 class d) ? the over-Sanskritized / stiff terms must
stay replaced
112 // on the AUDITED keys. Key-scoped (not a repo-wide ban) so legitimate marketing prose
elsewhere is not
113 // constrained, while still catching a silent revert of the exact bug.
114 describe("catalog content ? Hindi de-Sanskritization stays fixed (#77 class d)", () => {
115     const hi = load("hi");
116     const BANNED: Array<[string, string]> = [
117         ["grid.shots", "????????????"], // "experimental" calque
118         ["grid.shotsTitle", "????????????"],
119         ["grid.shotsTitle", "????????????"], // "under development" bureaucratese
120         ["query.support", "????????????"],
121         ["query.support", "????????????"],
122         ["grid.shotsConfidence", "????????"], // "faith" ? wrong sense for statistical
confidence
123         ["app.badge", "?????-????"], // "per-rally" calque
124     ];
125     for (const [key, term] of BANNED) {
126         it(`hi ${key} no longer contains the stiff "${term}"`, () => {
127             expect(hi).toHaveProperty(key);
128             expect(hi[key] ?? "").not.toContain(term);
129         });
130     }
131     it("hi keeps the friendlier replacements (???????? / ????? ????)", () => {
132         expect(hi["grid.shots"]).toContain("????????");
133         expect(hi["grid.shotsTitle"]).toContain("????????");
134         expect(hi["query.support"]).toContain("????????");
135         expect(hi["grid.shotsConfidence"]).toContain("???? ????");
136     });
137 });
138
139 // (5) Flagged mistranslations ? substring guard (.not.toContain) so a padded/wrapped

```

```

variant of the
140 // banned term ("????? ????????", a trailing space, ?) is still caught.
141 describe("catalog content ? flagged mistranslations stay fixed (kn/hi word choice)", ()
=> {
142     it("kn bulk.invert drops the 'rotate' mistranslation (????????)", () => {
143         expect(load("kn")["bulk.invert"]).not.toContain("????????");
144     });
145     it("kn result.dismiss drops the legalistic '????????'", () => {
146         expect(load("kn")["result.dismiss"]).not.toContain("????????");
147     });
148     it("hi result.dismiss drops the harsh '????'", () => {
149         expect(load("hi")["result.dismiss"]).not.toContain("????");
150     });
151 });
152

```

13. user (2026-06-22T16:10:16.362Z)

```

1 // CI guardrails that keep i18n "by design" (docs/archives/FRIENDLY_AND_MULTILINGUAL.md
$4):
2 // 1. KEY-PARITY ? every locale catalog has EXACTLY the `en` key set (the
extensibility proof:
3 // drop a stub locale and this lists every gap). Trivially green while only `en`
exists; it
4 // starts enforcing the moment a kn/hi catalog lands (B5/B6).
5 // 2. NO-BARE-STRINGS (attribute level, v1) ? user-facing component attributes
(placeholder /
6 // aria-label / title) must route through t(), never a hardcoded literal. A curated
start per
7 // the doc; an AST-level sweep of JSX text can extend it later.
8 import { describe, it, expect } from "vitest";
9 import { readdirSync, readFileSync } from "node:fs";
10 import { join } from "node:path";
11
12 // Catalogs are hoisted to repo-root /locales (D-CATALOG-HOME): web/src/i18n ->
../.../locales.
13 const LOCALES_DIR = join(import.meta.dirname, "..", "..", "..", "locales");
14 const SRC_DIR = join(import.meta.dirname, "..");
15
16 function flatten(obj: Record<string, unknown>, prefix = ""): string[] {
17     const keys: string[] = [];
18     for (const [k, v] of Object.entries(obj)) {
19         const key = prefix ? `${prefix}.${k}` : k;
20         if (v && typeof v === "object" && !Array.isArray(v)) keys.push(...flatten(v as
Record<string, unknown>, key));
21         else keys.push(key);
22     }
23     return keys.sort();
24 }
25
26 const loadKeys = (locale: string): string[] =>
27     flatten(JSON.parse(readFileSync(join(LOCALES_DIR, locale, "common.json"), "utf8")) as
Record<string, unknown>);
28
29 describe("i18n key-parity ? every locale has EXACTLY the en keys", () => {
30     const enKeys = loadKeys("en");
31     const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
32         .filter((d) => d.isDirectory() && d.name !== "en")
33         .map((d) => d.name);
34
35     it("en is the non-empty source catalog", () => {
36         expect(enKeys.length).toBeGreaterThan(20);
37     });
38
39     for (const loc of others) {

```

```

40     it(`${loc} matches en exactly (no missing, no extra keys)`, () => {
41         const keys = loadKeys(loc);
42         const missing = enKeys.filter((k) => !keys.includes(k));
43         const extra = keys.filter((k) => !enKeys.includes(k));
44         expect({ locale: loc, missing, extra }).toEqual({ locale: loc, missing: [], extra:
45     [] });
46     });
47     });
48
49     // Key-parity proves every locale has the same KEYS; this proves every locale's VALUE
keeps the same
50     // simple `{placeholder}` set as en ? so a translation can never silently drop a
{count}/{error}/{min}/
51     // {filename}/{n}/? and break interpolation. (ICU plural headers like `{count, plural,
?}` carry no
52     // simple {word} placeholder and match as empty on both sides, so they're correctly
ignored.)
53     describe("placeholder parity ? every locale preserves en's {placeholders}", () => {
54         function flatEntries(obj: Record<string, unknown>, prefix = ""): Record<string,
string> {
55             const out: Record<string, string> = {};
56             for (const [k, v] of Object.entries(obj)) {
57                 const key = prefix ? `${prefix}.${k}` : k;
58                 if (v && typeof v === "object" && !Array.isArray(v)) Object.assign(out,
flatEntries(v as Record<string, unknown>, key));
59                 else out[key] = String(v);
60             }
61             return out;
62         }
63         const placeholders = (s: string) => (s.match(/\{(\w+)\}/g) ?? []).sort();
64         const load = (loc: string) => flatEntries(JSON.parse(readFileSync(join(LOCALES_DIR,
loc, "common.json"), "utf8")));
65         const enE = load("en");
66         const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
67             .filter((d) => d.isDirectory() && d.name !== "en")
68             .map((d) => d.name);
69
70         for (const loc of others) {
71             it(`${loc} preserves every {placeholder} that en uses`, () => {
72                 const locE = load(loc);
73                 const mismatches: Array<{ key: string; en: string[]; got: string[] }> = [];
74                 for (const key of Object.keys(enE)) {
75                     if (!(key in locE)) continue; // missing keys are the key-parity test's job
76                     const a = placeholders(enE[key]);
77                     const b = placeholders(locE[key]);
78                     if (a.join(",") !== b.join(",")) mismatches.push({ key, en: a, got: b });
79                 }
80                 expect(mismatches).toEqual([]);
81             });
82         }
83     });
84
85     describe("es 3-form plural (CLDR one/many/other) ? MULTILINGUAL Phase A", () => {
86         // Spanish cardinal is {one, many, other}; every es ICU plural message MUST carry a
`many` arm
87         // (the `many` category fires on large/compact numbers) or intl-messageformat silently
falls back.
88         // These are the plural-bearing keys in the catalog.
89         const es = JSON.parse(readFileSync(join(LOCALES_DIR, "es", "common.json"), "utf8")) as
Record<string, Record<string, string>>;
90         const pluralKeys: Array<[string, string]> = [
91             ["footer", "count"],
92             ["footer", "floorSomeLead"],
93             ["selection", "count"],

```

```

94     };
95     for (const [ns, key] of pluralKeys) {
96         it(`es ${ns}.${key} carries one/many/other arms`, () => {
97             const msg = es[ns][key];
98             expect(msg, `${ns}.${key}`).toMatch(/\\bone\\s*\\/);
99             expect(msg, `${ns}.${key}`).toMatch(/\\bmany\\s*\\/);
100            expect(msg, `${ns}.${key}`).toMatch(/\\bother\\s*\\/);
101        });
102    }
103 });
104
105 describe("no-bare-strings ? user-facing component attributes go through t()", () => {
106     const FILES = [
107         "App.tsx",
108         "components/QueryBar.tsx",
109         "components/TapFilters.tsx",
110         "components/RallyGrid.tsx",
111         "components/SelectionFooter.tsx",
112         "components/CompileResult.tsx",
113         "components/LanguageSwitcher.tsx",
114         "components/IngestPanel.tsx",
115         "components/SetupWizard.tsx",
116     ];
117     // A hardcoded placeholder / aria-label / title with letters is a bare-string
118     regression ? it
119     // must be an expression ({t(...)}), not a quoted literal.
120     const BARE_ATTR = /\\b(?:placeholder|aria-label|title)\\s*=\\s*"^[^"]*[A-Za-z]{2}[^"]*"\\/g;
121
122     for (const f of FILES) {
123         it(`${f}: no hardcoded placeholder/aria-label/title`, () => {
124             const src = readFileSync(join(SRC_DIR, f), "utf8");
125             expect(src.match(BARE_ATTR) ?? []).toEqual([]);
126         });
127     }
128 });
129
130 describe("Indic fonts (B4) ? self-hosted OFL Noto, no runtime CDN", () => {
131     it("main.tsx bundles the OFL Noto Kannada + Devanagari fonts (self-hosted, not Google Fonts)", () => {
132         const main = readFileSync(join(SRC_DIR, "main.tsx"), "utf8");
133         expect(main).toMatch(/@fontsource-variable\\n\\oto-sans-kannada\\/);
134         expect(main).toMatch(/@fontsource-variable\\n\\oto-sans-devanagari\\/);
135     });
136     it("the font stack falls back to Noto for Kannada + Devanagari glyphs", () => {
137         const css = readFileSync(join(SRC_DIR, "index.css"), "utf8");
138         expect(css).toMatch(/Noto Sans Kannada Variable\\/);
139         expect(css).toMatch(/Noto Sans Devanagari Variable\\/);
140     });
141 });
142
143 // The Studio SPA must render correctly on an AIR-GAPPED appliance (GPU/CPU/RAM, no
144 internet) ?
145 // every asset (fonts, styles, scripts) is bundled local, nothing pulled from a
146 CDN/Google Fonts.
147 describe("offline / self-contained ? no external runtime assets", () => {
148     const CDN = /googleapis|gstatic|fonts\\.google|cdnjs|unpkg|jsdelivr|\\bcdn\\. [a-z]\\/i;
149     const FILES = [
150         "../index.html", // web/index.html
151         "main.tsx",
152         "index.css",
153     ];
154     for (const rel of FILES) {
155         it(`${rel}: no external font/CDN reference`, () => {
156             expect(readFileSync(join(SRC_DIR, rel), "utf8")).not.toMatch(CDN);
157         });
158     }
159 }

```



```

+++ b/locales/kn/common.json
@@ -91,7 +91,15 @@
    "fileHint": "? ???????.mp4 ????? .mov ????? ? ??? ??????? ????????, ??? KhelSutra
    ?????????? ??????????.",
    "orLink": "???? ?????? ?????? ????",
    "uploadingStatus": "????????? ??????????",
-   "uploadProgress": "????????? ??????"
+   "uploadProgress": "????????? ??????",
+   "compute": {
+     "legend": "?????? ?????? ??????????",
+     "local": "???? ??????????????????",
+     "cloud": "?????????????????",
+     "cloudCost": "?????? ?????????? ?????? GPU ?????? ?????????? ? ?????? ?????????????? ???, ???
    ?????? ?????????????? (????????????? ?????? ?????????? ?????? ??????????).",
+     "ack": "?????? ?????????????? ?????? ?????????????? ??? ??? ??????????.",
+     "ackRequired": "????????????? ?????? ?????? ?????????????? ??????????????"
+   }
  },
  "errors": {
    "networkError": "KhelSutra ??????????????????. ??? ??????????????????",

```

16. user (2026-06-22T16:10:24.011Z)

```
144:export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
```

17. assistant (2026-06-22T16:10:35.084Z)

Now let me verify two specific concerns. First, the jargon guard: the new strings use "GPU" ? let me confirm "GPU" is not in the banned jargon list (it isn't ? the list is engine/server/bucket/backend). Let me check the en `cloudCost` uses "GPU" and kn/hi keep "GPU" Latin. The guard only checks engine/server/bucket/backend, so GPU is fine. But let me double check the `i18n.resolvedLanguage` brand rule ? the new strings have no brand, so brand-present-where-en-uses-it is satisfied (en's compute.* keys don't include "KhelSutra").

Now the critical concurrency/stale-closure question. Let me confirm `getCapabilities` and check whether the `aria-pressed` toggle group and checkbox have proper labels. Let me read `getCapabilities`.

18. user (2026-06-22T16:10:35.215Z)

```

132
133    /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
134    export function highlightUrl(videoId: string, filename: string): string {
135        return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
136    }
137
138    /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
139    export async function getConfig(cid?: string): Promise<EngineConfig> {
140        return request<EngineConfig>("/config", { method: "GET", cid });
141    }
142
143    /** What THIS box can do ? drives the first-run wizard. GET /api/capabilities
(PACKAGING_PLAN P2c). */
144    export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
145        return request<EngineCapabilities>("/capabilities", { method: "GET", cid });
146    }
147
148    /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
149    export function minShotCount(cfg: EngineConfig): number | undefined {
150        return cfg.stitching?.min_shot_count;
151    }

```

19. assistant (2026-06-22T16:10:47.328Z)

Now let me trace the most important adversarial path carefully ? the ****stale compute on the upload path****. The key claim in the comment is "The arrow is recreated each render (useUpload reads onCompleteRef.current), so sendCompute is fresh."

Let me verify this reasoning precisely:

- ``useUpload(path) => submit(path, locale, sendCompute)`` ? this arrow is recreated on every render of ``IngestPanel``.
- Inside ``useUpload``, ``onCompleteRef.current = onComplete;`` runs on every render (line 25), so the ref always holds the latest arrow capturing the latest ``sendCompute``.
- On upload completion, ``onCompleteRef.current(res.path)`` (line 63) calls the latest arrow ? latest ``sendCompute``.

So far it captures the LATEST ``sendCompute`` at the time the upload ****completes****. Now let me trace the specific adversarial scenarios. The scenario "picks cloud, acks, picks file, switches back to local before upload completes" ? does it send local? Yes, because ``onCompleteRef`` is updated each render and the latest ``sendCompute`` would be ``local``. That's actually the ***safe*** direction (downgrade, never sends paid cloud silently).

But there's a subtler concern: ****the file input is NOT disabled when cloud is selected+acked**** ? ``inputsDisabled = busy || cloudBlocked``, and ``cloudBlocked`` is false once acked. That's the intended path. Let me probe the real bypass: ****does picking a file auto-submit, and is the input disabled at the right times?**** Let me run the tests and typecheck to ground the behavior.

20. user (2026-06-22T16:10:48.707Z)

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview",
  "typecheck": "tsc --noEmit",
  "test": "vitest run",
  "test:coverage": "vitest run --coverage"
},
"dependencies": {
  "@fontsource-variable/noto-sans-devanagari": "^5.2.8",
  "@fontsource-variable/noto-sans-kannada": "^5.2.9",
  "i18next": "^26.3.1",
  "i18next-browser-languagedetector": "^8.2.1",
  "i18next-icu": "^2.4.3",
  "react": "^19.0.0",
  "react-dom": "^19.0.0",
  "react-i18next": "^17.0.8"
},
"devDependencies": {
  "@tailwindcss/vite": "^4.0.0",
  "@testing-library/dom": "^10.4.1",
```

21. assistant (2026-06-22T16:10:55.202Z)

That package.json is at the ``web/`` root (the working dir is ``web``). Let me run the tests, typecheck, and lint to ground the behavior.

22. user (2026-06-22T16:10:58.942Z)

```
> khelsutra-web@0.0.0 typecheck
> tsc --noEmit
```

23. assistant (2026-06-22T16:11:01.178Z)

Typecheck passes. Now the tests:

24. user (2026-06-22T16:11:07.671Z)

```

stdout | src/components/build-error.test.tsx > A5: friendly build-error
mapping > LW2: the active UI locale is sent to /api/compile (drives the localized
reel watermark)
{"ts":"2026-06-22T16:11:07.368Z","level":"info","event":"api.response","cid":"d5f5b6ab-741c-4307-8668-7738ea6e005e","method":"POST","url":"/api/compile","status":200}

stdout | src/components/build-error.test.tsx > A5: friendly build-error
mapping > LW2: the active UI locale is sent to /api/compile (drives the localized
reel watermark)
{"ts":"2026-06-22T16:11:07.368Z","level":"info","event":"build.done","cid":"d5f5b6ab-741c-4307-8668-7738ea6e005e","video_id":"vidX","filename":"highlights.mp4","filepath":"/o/r.mp4","url":"/api/highlights/vidX/highlights.mp4"}

? src/components/build-error.test.tsx (4 tests)
783ms
? A5: friendly build-error mapping > a network / engine-down
compile failure shows the friendly 'is it running?' line 506ms
stdout | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-22T16:11:07.346Z","level":"info","event":"api.request","cid":"6910843f-04ca-4286-b00e-6e0699e95819","method":"GET","url":"/api/capabilities"}
{"ts":"2026-06-22T16:11:07.346Z","level":"info","event":"api.request","cid":"387c10af-b7cb-4298-838a-3ddc3f834a34","method":"GET","url":"/api/config"}
{"ts":"2026-06-22T16:11:07.369Z","level":"info","event":"api.request","cid":"dadb353a-9fdc-4680-8952-83aa3d1bb546","method":"GET","url":"/api/capabilities"}
{"ts":"2026-06-22T16:11:07.369Z","level":"info","event":"api.request","cid":"2451fb75-3f96-40bb-af25-1a298115eb11","method":"GET","url":"/api/config"}
{"ts":"2026-06-22T16:11:07.396Z","level":"info","event":"api.request","cid":"f591839d-328f-4e5d-82e0-0755028aaa69","method":"GET","url":"/api/capabilities"}
{"ts":"2026-06-22T16:11:07.397Z","level":"info","event":"api.request","cid":"428957ac-30fe-4b32-881a-55ec4c746238","method":"GET","url":"/api/config"}

stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-22T16:11:07.358Z","level":"error","event":"api.network_error","cid":"6910843f-04ca-4286-b00e-6e0699e95819","method":"GET","url":"/api/capabilities","error":"Error: no engine in test"}
{"ts":"2026-06-22T16:11:07.358Z","level":"error","event":"api.network_error","cid":"387c10af-b7cb-4298-838a-3ddc3f834a34","method":"GET","url":"/api/config","error":"Error: no engine in test"}
{"ts":"2026-06-22T16:11:07.381Z","level":"error","event":"api.network_error","cid":"dadb353a-9fdc-4680-8952-83aa3d1bb546","method":"GET","url":"/api/capabilities","error":"Error: no engine in test"}
{"ts":"2026-06-22T16:11:07.382Z","level":"error","event":"api.network_error","cid":"2451fb75-3f96-40bb-af25-1a298115eb11","method":"GET","url":"/api/config","error":"Error: no engine in test"}
{"ts":"2026-06-22T16:11:07.413Z","level":"error","event":"api.network_error","cid":"f591839d-328f-4e5d-82e0-0755028aaa69","method":"GET","url":"/api/capabilities","error":"Error: no engine in test"}
{"ts":"2026-06-22T16:11:07.413Z","level":"error","event":"api.network_error","cid":"428957ac-30fe-4b32-881a-55ec4c746238","method":"GET","url":"/api/config","error":"Error: no engine in test"}
{"ts":"2026-06-22T16:11:07.413Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-22T16:11:07.413Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)

```

```
{"ts":"2026-06-22T16:11:07.413Z","level":"warn","event":"wizard.capabilities_unavailable","cid":"f591839d-328f-4e5d-82e0-0755028aaa69","error":"ApiError: Network error calling GET /api/capabilities"}
{"ts":"2026-06-22T16:11:07.413Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}
```

```
? src/i18n/i18n.test.tsx (17 tests)
823ms
```

```
? i18n scaffold (B1) > renders user-facing copy from the en
catalog (not raw keys) 412ms
```

```
Test Files 24 passed (24)
Tests 226 passed (226)
Start at 21:41:02
Duration 4.56s (transform 1.43s, setup 846ms, collect 8.62s, tests 5.33s,
environment 31.36s, prepare 4.88s)
```

25. assistant (2026-06-22T16:11:21.448Z)

All 226 tests pass. Now let me dig into the most adversarial scenario that the tests do NOT cover ? the **upload auto-submit bypass**. Let me carefully trace whether a user can dispatch `compute=cloud` without acking, or whether state state can leak, by examining the precise React state-update timing.

The critical scenario: **User selects cloud, does NOT ack, then picks a file via the file input.**

- When `compute === "cloud" && !cloudAck`, `cloudBlocked = true`, so `inputsDisabled = true`. The file `<input disabled={inputsDisabled}>` ? disabled. So the file picker can't fire `onFile`. Good ? that path is blocked.

Now the **acked then file-pick** scenario: ack ? `cloudBlocked = false` ? input enabled ? pick file ? upload ? on complete, `submit(path, locale, sendCompute)` where `sendCompute = "cloud"`. Correct.

Now let me probe a real subtle bug: **what if the user picks a file while local, the upload is in flight (busy=true), and during the upload they switch to cloud + ack?** On upload completion, `onCompleteRef.current` holds the LATEST arrow, capturing the latest `sendCompute = "cloud"` ? so it would submit `compute=cloud` for a file the user picked while on local. But the toggle buttons are `disabled={busy}` during upload, so the user CANNOT switch compute mid-upload. Let me verify the cost checkbox is also `disabled={busy}` ? yes (line 122). So mid-upload, neither the toggle nor the checkbox can change. Good.

But wait ? there's a more subtle gap. Let me check: the compute toggle buttons use `disabled={busy}`, but `busy = ingestBusy || upload.busy`. During the **engine polling phase** (`ingestBusy` true, after upload handed off), can compute change? No, still disabled. Good.

Let me now probe the **real** potential bug: the cost-gate bypass via the file path when cloud is selected but **the ack checkbox interaction races**. Let me look at whether `onFile` itself re-checks `cloudBlocked`. It does NOT:

26. user (2026-06-22T16:11:22.842Z)

```
59      const onSubmitUri = (e: FormEvent) => {
60        e.preventDefault();
61        if (cloudBlocked) return;
62        submit(uri, locale, sendCompute);
63      };
64      const onFile = (e: ChangeEvent<HTMLInputElement>) => {
65        const file = e.target.files?.[0];
66        if (file) upload.start(file);
67        e.target.value = ""; // let re-picking the same file fire change again
68      };
```

27. assistant (2026-06-22T16:11:37.774Z)

Here's the asymmetry: `onSubmitUri` has a belt-and-suspenders `if (cloudBlocked) return;` guard, but `onFile` does NOT have an equivalent guard. The only thing preventing a file-pick while `cloudBlocked` is the `disabled={inputsDisabled}` attribute on the `

For a native disabled file input, the change event genuinely can't fire (the OS file dialog can't open). So in practice this is belt-only without suspenders. It's defensible since a disabled native input cannot trigger `onChange`. Not a real bug, but worth noting the asymmetry. Let me check whether there's any way the file input becomes enabled while cloud is unacked.

```
`inputsDisabled = busy || cloudBlocked`. `cloudBlocked = cloudAvailable && compute === "cloud" && !cloudAck`. When `compute==="cloud"` and `!cloudAck`, `cloudBlocked` is true ? input disabled. There's no render window where compute is cloud, unacked, and the input is enabled. So no bypass.
```

Now let me check the **accessibility** concerns precisely. The checkbox has a `` wrapping it with text ? accessible name OK. The toggle buttons have `aria-pressed` and visible text. Let me verify the existing panel convention for the 44px target and compare. The toggle buttons use `min-h-11` (44px) ? matches convention.

Let me check one real a11y concern: the `compute-cost` paragraph and the checkbox ? is the cost note programmatically associated? Not required for WCAG AA necessarily. Let me check the `ackRequired` amber text contrast and whether `aria-live` would help, but that's an enhancement not a regression.

Let me now check a genuinely important thing: **does the `inputsDisabled` change cause the URI submit button to look disabled but the form `onSubmit` still firing?** When `cloudBlocked`, the submit button is `disabled` AND `onSubmitUri` early-returns. Double-covered. Good.

Now the **most important real correctness check**: backward-compat / default-OFF. When `cloud\_available` is false or capabilities fails:

```
- `cloudAvailable` stays `false`.
- `sendCompute = cloudAvailable ? compute : undefined` ? `undefined`.
- `processVideo(uri, cid, locale, undefined)` ? `JSON.stringify({ video_path, locale, compute: undefined })` ? `compute` key dropped.
```

Let me verify there are no OTHER callers of `submit` or `processVideo` that might now pass something unexpected, and check the `site/public/locales` mirror is in sync (the guard reads repo-root `/locales`, but the site mirror must match for runtime).

28. user (2026-06-22T16:11:41.778Z)

```
=== diff repo-locales vs site mirror for the new compute block (en) ===
HEAD: en repo==site identical
--- working tree: are the new compute keys identical across repo + site mirror for all 6
locales? ---
en: IDENTICAL
kn: IDENTICAL
hi: IDENTICAL
es: IDENTICAL
da: IDENTICAL
id: IDENTICAL
```

29. assistant (2026-06-22T16:11:44.688Z)

All 6 locale catalogs are identical between repo-root and site mirror. Now let me check all callers of `submit` and `processVideo` to ensure no other caller is affected, and verify the `useUpload` callers.

30. user (2026-06-22T16:11:46.088Z)

```
src\hooks\useUpload.ts:20:export function useUpload(onComplete: (path: string) => void) {
src\hooks\useIngestJob.test.ts:37:    await vi.advanceTimersByTimeAsync(0); // flush
```

```

processVideo ? schedule the first poll
src\hooks\useIngestJob.test.ts:59:    act(() => result.current.submit("  "));
src\hooks\useIngestJob.test.ts:77:    result.current.submit("gs://b/x.mp4");
src\hooks\useIngestJob.test.ts:95:    result.current.submit("gs://b/x.mp4");
src\hooks\useIngestJob.test.ts:112:    result.current.submit("gs://b/x.mp4");
src\hooks\useIngestJob.test.ts:123:    result.current.submit("ftp://nope");
src\hooks\useIngestJob.test.ts:141:    result.current.submit("gs://b/x.mp4");
src\hooks\useIngestJob.test.ts:160:    result.current.submit("gs://b/x.mp4");
src\hooks\useIngestJob.ts:10:import { getJob, processVideo } from "../api/client";
src\hooks\useIngestJob.ts:119:    const resp = await processVideo(uri, cid, locale,
compute);
src\hooks\useUpload.test.ts:8:import { useUpload } from "../useUpload";
src\hooks\useUpload.test.ts:40:describe("useUpload", () => {
src\hooks\useUpload.test.ts:45:    const { result } = renderHook(() => useUpload(onComplete));
src\hooks\useUpload.test.ts:57:    const { result } = renderHook(() => useUpload(onComplete));
src\hooks\useUpload.test.ts:67:    const { result } = renderHook(() => useUpload(onComplete));
src\api\client.ts:85:export async function processVideo(
src\api\client.ts:198:/** Finalize: returns {path, video_id} ? feed `path` to processVideo()
[uploads.py:123]. */
src\api\locale.test.ts:5:import { compileReel, processVideo } from "../client";
src\api\locale.test.ts:36: it("processVideo includes the locale when given, omits it when
absent", async () => {
src\api\locale.test.ts:39:    await processVideo("gs://b/x.mp4", "cid", "hi");
src\api\locale.test.ts:42:    await processVideo("gs://b/y.mp4", "cid");
src\components\IngestPanel.tsx:12:import { useUpload } from "../hooks/useUpload";
src\components\IngestPanel.tsx:53: // The arrow is recreated each render (useUpload reads
onCompleteRef.current), so sendCompute is fresh.
src\components\IngestPanel.tsx:54: const upload = useUpload((path) => submit(path, locale,
sendCompute));
src\components\IngestPanel.tsx:62:    submit(uri, locale, sendCompute);
src\components\QueryBar.test.tsx:48:    fireEvent.submit(applyBtn().closest("form"));

```

31. assistant (2026-06-22T16:11:50.153Z)

`IngestPanel` is the only caller of `useUpload` and the only place `submit(...)` is invoked with a compute arg. Let me check the existing `processVideo` locale test to confirm the `compute: undefined` omission is consistent with how `locale: undefined` is already tested ? to verify the JSON.stringify-drops-undefined claim holds.

32. user (2026-06-22T16:11:51.257Z)

```

30      vi.stubGlobal("fetch", f);
31      await compileReel("vid", [1], "r.mp4");
32      expect(bodyOf(f)).toEqual({ video_id: "vid", segment_ids: [1], output_filename:
"r.mp4" });
33      expect("locale" in bodyOf(f)).toBe(false);
34    });
35
36    it("processVideo includes the locale when given, omits it when absent", async () => {
37      const f = ok({ job_id: "j", status: "queued", poll: "/api/jobs/j" });
38      vi.stubGlobal("fetch", f);
39      await processVideo("gs://b/x.mp4", "cid", "hi");
40      expect(bodyOf(f)).toEqual({ video_path: "gs://b/x.mp4", locale: "hi" });
41
42      await processVideo("gs://b/y.mp4", "cid");
43      expect(bodyOf(f)).toEqual({ video_path: "gs://b/y.mp4" });
44    });
45  });
46

```

33. assistant (2026-06-22T16:12:05.726Z)

This test confirms the JSON.stringify-drops-undefined behavior: `processVideo("gs://b/y.mp4", "cid")` produces `{ video\_path }` with no `locale` and no `compute` key. So the backward-compat

claim is solidly verified by an existing (still-passing) test ? when `compute` is undefined, the body is byte-identical to before.

Now let me probe one more genuinely adversarial concern: the **`compute` value space**. The SPA's `compute` state is typed `Compute = "local" | "cloud"` and `setCompute` is only ever called with `opt` from ["local","cloud"] as const`. sendCompute is cloudAvailable ? compute : undefined. So the SPA only ever sends "local", "cloud", or omits. Confirmed ? no path sends null, "", or any other value.`

Now let me double-check one subtle React 19 concern with the capabilities `useEffect` and act() warnings, and whether a rejected/garbage capabilities response crashes. The .catch` handles rejection. But what if capabilities resolves with garbage (e.g., deployment` is a string)? c.deployment?.cloud_available === true` ? if c.deployment` is a string, .cloud_available` is undefined, === true` is false ? picker hidden, no crash. If c` itself is null... getCapabilities` returns the parsed JSON typed as EngineCapabilities`; c.deployment?.` would throw if c` is null. Let me check: can the engine return null` body with 200? request` does res.json() ? if body is literally null, c` is null, and c.deployment` throws TypeError inside .then, which is NOT caught by .catch` (the throw is in the fulfillment handler).`

Let me verify this precisely. In a promise chain `p.then(onFulfilled).catch(onRejected)` , if onFulfilled` throws, the .catch` DOES catch it (it's .then(onFulfilled).catch(...)` , so the catch is downstream). Let me re-read:`

```
```js
getCapabilities()
 .then((c) => alive && setCloudAvailable(c.deployment?.cloud_available === true))
 .catch((err) => log("warn", ...));
```
```

The `.catch` is chained after .then, so a throw inside the .then` callback IS caught. So even c === null` (TypeError on .deployment` ) is caught and logged. No unhandled rejection, no crash. Graceful degradation holds.`

Let me verify the `act()/async` handling won't leave a dangling state update after unmount ? the alive` flag guards setCloudAvailable`. Good. Now let me run lint to match CI.`

34. user (2026-06-22T16:12:10.173Z)

Oops! Something went wrong! :(

ESLint: 10.5.0

ESLint couldn't find an `eslint.config.(js|mjs|cjs)` file.

From ESLint v9.0.0, the default configuration file is now `eslint.config.js`.

If you are using a `.eslintrc.*` file, please follow the migration guide to update your configuration file to the new format:

<https://eslint.org/docs/latest/use/configure/migration-guide>

If you still have problems after following the migration guide, please stop by <https://eslint.org/chat/help> to chat with the team.

exit: 0

35. assistant (2026-06-22T16:12:12.490Z)

No eslint config in `web/`. Let me check if there's a lint script or config elsewhere.`

36. user (2026-06-22T16:12:14.949Z)

---root---

--- lint script in any package.json ---

```
./site/node_modules/@asamuzakjp/dom-selector/package.json
./site/node_modules/@asamuzakjp/generational-cache/package.json
./site/node_modules/@asamuzakjp/nwsapi/package.json
./site/node_modules/@bramus/specificity/package.json
./site/node_modules/@exodus/bytes/package.json
./site/node_modules/css-tree/package.json
./site/node_modules/data-urls/package.json
./site/node_modules/entities/package.json
./site/node_modules/html-encoding-sniffer/package.json
./site/node_modules/jsdom/package.json
```

37. assistant (2026-06-22T16:12:21.124Z)

No project-level ESLint is configured for `web/` (only `tsc` + vitest are the gates, both green). Lint isn't part of this project's CI for the web app.

Now let me do a final adversarial check on the **brand-Latin guard** with the new keys, and confirm the new `cloudCost` strings with "GPU" don't trip any guard. Let me run just the i18n guard tests explicitly to confirm they pass with the new keys.

38. user (2026-06-22T16:12:24.281Z)

```
RUN v3.2.6 C:/Users/avidu/Projects/khelsutra/web

? src/i18n/hygiene.test.ts (29 tests)
17ms
? src/i18n/catalog-invariants.test.ts (24 tests)
21ms

Test Files  2 passed (2)
Tests      53 passed (53)
Start at   21:42:23
Duration   520ms (transform 61ms, setup 23ms, collect 81ms, tests 38ms,
environment 0ms, prepare 340ms)
```

39. assistant (2026-06-22T16:12:50.650Z)

All i18n guards pass. Let me verify one final subtle thing about the jargon guard ? it checks "GPU" isn't in the banned list. Confirmed banned = engine/server/bucket/backend only; "GPU" is allowed everywhere. The kn/hi `cloudCost` keeps "GPU" in Latin (correct ? not transliterated), and the brand "KhelSutra" is absent from all new keys in all locales, so the positive brand guard (present-where-en-uses-it) is satisfied since en's compute keys don't carry the brand.

Let me make one last check on a real-world edge: **does the `cloudCost` paragraph text containing "GPU" (an English acronym embedded in kn/hi) violate the hygiene placeholder-parity?** No placeholders in these strings. And the no-bare-strings guard only checks `placeholder`/`aria-label`/`title` attributes in JSX ? the new toggle buttons use `aria-pressed` (boolean, not text) and `{t(...)}` for labels. No hardcoded attribute strings added. Confirmed green above.

I have thoroughly traced every requested attack path. Here are my findings.

Adversarial review: compute picker on the ingest screen

Verdict: No blocker/major/minor correctness, security, ally, i18n-guard, or backward-compat bugs found. The change is sound. Verified locally: `tsc --noEmit` clean, full `vitest` suite green (226 passed), i18n guards green (53 passed), all 6 locale catalogs identical between `locales/` and `site/public/locales/`. (No ESLint config exists for `web/` ? `tsc`+vitest are the actual gates.)

Per-probe verdicts:

****1. Bypass the cost gate ? SAFE.**** No path dispatches `compute=cloud` without an ack.
- `cloudBlocked = cloudAvailable && compute==="cloud" && !cloudAck` drives `inputsDisabled`, which disables both the file `<input>` (`web/src/components/IngestPanel.tsx:153`) and the URI input+submit button (`:171`, `:176`). A native disabled file input cannot open the dialog, so `onFile` can't fire while cloud is unacked.
- The URI form has belt-and-suspenders: button `disabled` ****plus**** `if (cloudBlocked) return;` in `onSubmitUri` (`:61`).
- Note (not a bug): `onFile` (`:64`) lacks the symmetric `if (cloudBlocked) return;` guard that `onSubmitUri` has ? it relies solely on the `disabled` attribute. Since a disabled native file input genuinely cannot emit `change`, there's no exploitable window, so I'm not flagging it. A one-line guard would make it consistent with the URI path if you want defense-in-depth.

****2. Stale compute on the upload path ? SAFE, and the safe direction.**** `useUpload((path) => submit(path, locale, sendCompute))` (`:54`) is recreated every render; `useUpload` writes `onCompleteRef.current = onComplete` every render (`web/src/hooks/useUpload.ts:25`) and calls `onCompleteRef.current(res.path)` on completion (`:63`), so it always uses the ****latest**** `sendCompute`. The `"cloud?ack?pick?switch back to local?completes"` case therefore sends `local` (a downgrade ? never silently dispatches a paid cloud job). The reverse `"local?pick?switch to cloud mid-upload"` cannot happen: both toggle buttons (`:102`) and the ack checkbox (`:122`) are `disabled={busy}` during upload and polling.

****3. Default-OFF / backward compat ? VERIFIED byte-identical.**** With `cloud_available` false or capabilities failing, `cloudAvailable` stays false ? `sendCompute = undefined` (`:47`) ? `JSON.stringify({video_path, locale, compute: undefined})` drops `compute`. The existing, still-passing test `web/src/api/locale.test.ts:42-43` proves an undefined trailing arg yields `{video_path}` only. Picker is not rendered (`:91`).

****4. Graceful degradation ? SAFE.**** A rejected capabilities call hits `.catch` (`:41`). A garbage 200 body (e.g. `deployment` as a string, or even `c === null`) throws inside the `.then` callback, which is caught by the chained `.catch` ? no unhandled rejection, no crash. The `alive` flag (`:38`, `:43`) guards against post-unmount state updates.

****5. Accessibility ? no regression.**** Toggle buttons carry `aria-pressed` + visible `{t(...)}` text and `min-h-11` (44px), matching panel convention. The checkbox is wrapped in a `<label>` with text (accessible name OK). No new hardcoded `placeholder`/`aria-label`/`title` literals (the no-bare-strings guard covers `IngestPanel.tsx` and passes).

****6. `compute` value space ? SAFE.**** `compute` state is typed `"local" | "cloud"`, only ever set from `["local", "cloud"]` as const, and `sendCompute` is `compute` or `undefined`. The SPA can only ever send `"local"`, `"cloud"`, or omit the key ? never `null`/`""`/other.

****i18n guards ? clean.**** New `ingest.compute.` keys carry no brand, so the "Latin KhelSutra present where en uses it" rule is vacuously satisfied. `cloudCost` uses "GPU", which is ****not**** in the banned engine/server/bucket/backend jargon list, and kn/hi keep "GPU" in Latin (not transliterated). Key-parity and placeholder-parity hold across all 6 locales + the site mirror.

Relevant files: `C:\Users\avidu\Projects\kheLSutra\web\src\components\IngestPanel.tsx`,
`web\src\hooks\useUpload.ts`, `web\src\hooks\useIngestJob.ts`, `web\src\api\client.ts`,
`web\src\api\types.ts`.